

TakeSeat

Technical Documentation

19 de maio de 2026

Índice

1. README
2. Product › Vision
3. Product › Features
4. Design System › Overview
5. Design System › Tokens
6. Design System › Components
7. Design System › Patterns
8. Architecture › Overview
9. Architecture › Components
10. Backend › Backend Architecture
11. Backend › Api Patterns
12. Frontend › Frontend Architecture
13. Frontend › State Management
14. Business Rules › Queue Rules
15. Business Rules › Customer Rules
16. Business Rules › Subscription Rules
17. Database › Schema Overview
18. Database › Entities › Restaurants
19. Database › Entities › Customers
20. Database › Entities › Waitlist Entries
21. Database › Entities › Subscriptions
22. Database › Entities › Users
23. Flows › Authentication Flow
24. Flows › Billing And Trial Flow
25. Flows › Queue Flow
26. Flows › Reporting Flow
27. Flows › User Management Flow
28. Flows › Whatsapp Flow
29. Integrations › Whatsapp

- 30. Integrations › Stripe
- 31. Decisions › ADR 001 Whatsapp Provider
- 32. Decisions › ADR 002 I18n Strategy
- 33. Decisions › ADR 003 Multi Tenant Model
- 34. Security › Security Overview
- 35. Ai Context › Agent Guidelines
- 36. Ai Context › System Context
- 37. Ai Context › Domain Glossary
- 38. Setup › Local Setup
- 39. Guides › Email Service Runbook
- 40. Troubleshooting › Troubleshooting Guide
- 41. Reference › Api Reference
- 42. Infrastructure › Infra Overview
- 43. Infrastructure › Deployment

 docs/README.md

TakeSeat Technical Documentation Map

Overview

This index serves as the master navigator ("single source of truth") for the TakeSeat waitlist system documentation, mapping business rules, system architecture, database structure, and integration specifications.

Responsibilities

- Map all documentation directories for engineering, product, and AI agents.
- Establish cross-references across all functional areas.

Architecture / Flow

[Diagrama Mermaid — ver fonte original]

Documentation Directories

1. Product & Design System

- **Product Specs:** Product modules, scope boundaries, and dashboard features.
- **Design Foundations:** Terracotta palette, typography, visual tokens, and responsive mobile-first guidelines.
- **UI Component Kit:** Reusable components, state indicators, layout shells, and responsive controls.

2. Architecture & Codebase Design

- **System Architecture Overview:** Multi-tenant logical isolation, stateless Lambda serverless design, and communication diagrams.
- **System Components:** Core technology stacks, migration runners, and backend domain service separation.
- **Backend Architecture:** Project layouts, Express controller routes, database service logic, and Zod validator schemas.
- **Frontend Architecture:** React workspace layout, route gates, context state managers, and locale systems.

3. Business Rules & Operations

- **Queue Rules:** Wait time logic, queue positioning rules, status workflows, and return timers.
- **Customer Rules:** CRM indexing constraints, validation formats, and activity audit histories.
- **Subscription Rules:** Free trials, pricing plan gates, billing states, and access limits.

4. Database & Models

- **Database Schema Overview:** Relationship diagrams, Prisma schemas, index optimization, and seed configurations.
- **Restaurant Tenant Model:** Settings fields, localized timezones, and configuration structures.
- **Customer Entity:** CRM columns, DDI numbers, and visit counters.
- **Waitlist Entry Entity:** Party sizes, status state machines, and obfuscated tokens.
- **Subscriptions Entity:** Billing status fields, payment flags, and trial windows.

5. Flows & Integrations

- **Authentication Flow:** Staff credentials verification, JWT issuance, and guest tracking access tokens.
- **Billing & Trial Flow:** State transitions, trial timeouts, and payment processing gates.
- **Queue Lifecycle Flow:** Joining queues, status updates, calls, and seating actions.
- **Reporting & KPIs Flow:** Analytics queries, performance computations, and exports.
- **User Management Flow:** Operator provisioning, roles checking, and password modifications.

- **WhatsApp Notification Flow:** Trigger schedules, template compiler runs, and delivery updates.
- **WhatsApp Integration:** Z-API configurations, template parameters, and rate-limits.
- **Stripe Billing Integration:** Webhook endpoints, Checkout redirections, and payment sync.

6. Operations & Guidelines

- **Security Overview:** JWT verification, logical isolation rules, password hashing, and transport safety.
- **AI Agent Guidelines:** Code invariants, viewport locks, and validation rules.
- **System Context Diagram:** Actor relationships and external bounds.
- **Domain Glossary:** Definitions of entities, calculations, and terms.
- **Local Setup Guide:** Docker commands, Prisma seed, and dev server launches.
- **Troubleshooting Guide:** Resolution workflows for connection, Z-API, and webhook errors.
- **API Reference:** REST paths, JSON request body rules, and response objects.
- **Architectural Decisions:** Records of decisions, alternatives, and trade-offs.

Related Documents

- [Features List](#)
- [System Architecture Overview](#)

 docs/product/vision.md

Product Vision

Overview

TakeSeat is a modern, responsive, and serverless queue management SaaS designed for high-turnover establishments (such as restaurants, bistros, and bars). It eliminates physical waiting lines, reduces customer walk-away rates, and optimizes front-of-house operations.

Responsibilities

The product is responsible for:

- Providing an intuitive mobile-first hostess panel to manage party queue entries.
- Offloading physical queues to a virtual waiting room with automated notifications.
- Informing customers dynamically of their position and estimated waiting time.
- Collecting wait metrics to help administrators optimize table turnover.

Architecture / Flow

1. **Entry:** Customer arrives at the establishment; the hostess records their name, party size, and phone number.
2. **Waiting Room:** Customer receives a WhatsApp notification with a link to their real-time position page. They are free to walk around nearby.
3. **Seating Call:** When a table is ready, the hostess triggers a call action, notifying the customer to return.
4. **Seated:** Customer is seated, ending their wait cycle, and their history is logged for CRM and analytics.

Rules

- **Mobile-First Orientation:** Front-of-house operators run the interface on tablets or mobile phones, while customers check their status solely on their mobile browsers.
- **LATAM Optimization:** The default country DDI is "+55" (Brazil), with phone number formats pre-formatted to handle local carrier changes (such as 9-digit mobile numbering). However, the system is multi-language and supports standard international formats.
- **SaaS Plan Alignment:** Advanced notification features and text personalization are gated behind the paid PRO plan. Trial status expires after 7 days, prompting the business to configure billing.

Edge Cases

- **No-Show Customers:** If a customer does not return within the establishment's custom threshold after being called, the entry status is marked as **NO_SHOW**.
- **Dynamic Viewport Variations:** The customer's mobile view must adapt seamlessly to varying mobile browser UI frames (like Chrome and Safari's address bar collapses) using dynamic viewport sizing (**dvh**).

Technical Notes

- Built using React on the frontend and an Express REST API served via AWS Lambda.
- Direct database modeling through Prisma, utilizing Aurora Serverless v2 for scale.

Related Documents

- [Features List](#)
- [Queue Rules](#)
- [System Context](#)

 docs/product/features.md

Product Features

Overview

TakeSeat provides a consolidated suite of features split across queue management, customer CRM, metrics, customization settings, and multi-tenant billing.

Responsibilities

Each component has specific operational boundaries:

- **Waitlist Operations:** Enables addition, modification, status transition, and cancellation of queue entries.
- **Customer Directory:** Centralizes visitor history, visits counts, and persistent opt-in preferences.
- **Analytics & Reporting:** Summarizes the last 7 days of performance (throughput and average wait times).
- **Billing & Subscription:** Governs gates, billing statuses, Stripe-integrated checkouts, and trial states.
- **Settings & WhatsApp Configuration:** Handles message templates, trigger rules, and general business information.

Architecture / Flow

- **Operator Panel:** Hostess interacts with `Waitlist` page → state updates via Axios requests → WebSocket (optional/future) broadcast or React Query invalidation refetches the state.
- **Customer View:** Customer tracks queue status via a public page using the `waitlistEntryId`.

- **System Settings:** Admin controls user credentials, configures messaging parameters, and manages Stripe billing checkout sessions.

Rules

- **Access Control Roles:**
 - **ADMIN** : Full access to settings, user management, reports, and waitlist.
 - **MANAGER** : Full access to waitlist, customers, and reports; cannot manage users or billing configurations.
 - **HOSTESS** : Operational access only; can only view and manage waitlist entries.
- **Active Subscription Constraint:** If the subscription state is **PAST_DUE** or **CANCELED** , waitlist operations are blocked until payment is resolved.
- **i18n Namespace Splitting:** Translation keys must be modularized into namespaces: **auth** , **common** , **waitlist** , **settings** , **reports** , **nav** , **profile** .

Edge Cases

- **Pickup Orders Toggle:** The database structure contains fields for pickup orders, but frontend support is deprecated. Only the Waitlist is active in the interface.
- **Invalid WhatsApp Numbers:** When notifications fail due to non-existent numbers, Z-API returns a webhook status which must be logged in **WhatsAppMessageLog** to notify the hostess of delivery failure.

Technical Notes

- Dual-axis line charts in reports are rendered using Recharts (**recharts**).
- Locale translation files are hosted at **frontend/src/locales/{lang}/{namespace}.json** .

Related Documents

- [Product Vision](#)
- [Subscription Rules](#)
- [Stripe Integration](#)
- [WhatsApp Integration](#)

 docs/design-system/overview.md

Design System Overview

Overview

The TakeSeat Design System (version 2.0.0) is a modern, scalable design language built on semantic tokens and inspired by modern visual interfaces. It is defined by a sophisticated indigo brand aesthetic that projects trust, clarity, and professionalism for restaurant operators and their guests.

Responsibilities

This documentation layer is responsible for:

- Enforcing structural visual consistency across the entire application interface.
- Outlining the Core Design Principles (Clarity, Progressive Disclosure, Trust, Performance, Accessibility, and Professionalism).
- Specifying cross-disciplinary patterns to accelerate developer implementation while staying accessible.

Architecture / Flow

1. **Design Tokens:** Basic values (primitives) feed into semantic meanings (surfaces, borders, text states) which then style components.
2. **Component Implementation:** Components utilize standard tokens directly via vanilla CSS variables or Tailwind CSS class configurations.
3. **Application Layout:** Pages assemble components to form unified structures.

Rules

- **Aesthetic Direction:** Deep indigo brand accents are balanced with cool slate neutrals and secondary violet details to preserve a professional, uncluttered layout.
- **Mobile First:** All touch target interactions, alignments, and navigation behaviors must scale dynamically to small mobile screens.
- **WCAG AA Compliance:** All text and background combinations must satisfy the WCAG AA minimum contrast ratio (4.5:1 for normal text, 3:1 for large text/icons).

Edge Cases

- **Dark Mode:** Dark mode is not a simple color inversion. It must reduce contrast for text (e.g., using `neutral-50` rather than pure white) and increase color saturation to maintain visibility.
- **Reduced Motion:** Motion animations must respect OS preferences by disabling transitions when `@media (prefers-reduced-motion: reduce)` is true.

Technical Notes

- The standard typography font family is **Inter**.
- Standard icon weights use a stroke width of 2px.

Related Documents

- [Design Tokens](#)
- [Design Components](#)
- [Design Patterns](#)

 docs/design-system/tokens.md

Design Tokens

Overview

Design Tokens are the visual building blocks of the TakeSeat visual architecture. They separate structural specs from concrete UI implementations, divided into Primitives, Semantics, and Components tokens.

Responsibilities

- Centralizing raw specs (color hashes, spacing scales, border radiuses, text styles).
- Mapping theme-independent semantic rules for light and dark mode operations.
- Outlining component-specific overrides (Buttons, Inputs, Cards, Modals).

Architecture / Flow

Primitive Value (raw hex/px) → Semantic Token (light/dark context) → Component Variable → HTML/CSS styling rendering .

Rules

- **Color Scales:**
 - **Neutral (Slate):** `neutral-0` (#FFFFFF) to `neutral-950` (#020617).
 - **Primary Brand:** `indigo-50` (#EEF2FF) to `indigo-950` (#1E1B4B).
 - **Accent:** `violet-50` (#F5F3FF) to `violet-950` (#2E1065).

- **Feedback:** `green` (Success), `red` (Error), `amber` (Warning), `sky` (Info).
- **8pt Spacing Scale:** All spacing uses multiples of 4px/8px: `spacing-1` (4px), `spacing-2` (8px), `spacing-3` (12px), `spacing-4` (16px), up to `spacing-64` (256px).
- **Corner Radiuses:**
 - Buttons: `radius-md` (8px).
 - Cards: `radius-lg` (12px).
 - Modals: `radius-xl` (16px).
 - Generic: `radius-sm` (4px), `radius-base` (6px), `radius-2xl` (24px).
- **Typographic Scale:** Font sizes scale from `text-xs` (12px) to `text-7xl` (72px) with strict line-height associations.

Edge Cases

- **Strict Component Corner Radius Warning:** Never use arbitrary corner radii. A button must always use `radius-md` (8px). A card must always use `radius-lg` (12px).
- **Focus Rings:** Interactive elements in focused states must use `border-focus` (`indigo-500` / #6366F1) combined with an outer glow outline-offset.

Technical Notes

- Shadows are configured with indigo hues for primary states (e.g., `shadow-indigo-sm`).
- Animations use easing transitions of `200ms` (fast) to `300ms` (base default).

Related Documents

- [Design System Overview](#)

- [Design Components](#)
- [Design Patterns](#)

 docs/design-system/components.md

Design System Components

Overview

This document specifies the standard reusable UI components in TakeSeat, defining their visual variants, code structures, sizing, states, and accessibility details.

Responsibilities

- Define exact specs for: **Button, Input, Card, Modal/Dialog, Badge, Toast/Notification, Table.**
- Standardize code parameters and design states to guide React component development.
- Enforce built-in keyboard navigation rules and screen reader compatibility.

Architecture / Flow

Components receive state variables or hooks → compile styles utilizing token-mapped variables → output WCAG-compliant accessible HTML.

Rules

- **Button Variants:**
 - *Primary (Indigo):* Brand CTA. Uses **indigo-500** background (#6366F1), white text, **shadow-indigo-sm**. Limit to one per screen.
 - *Secondary:* Outline style. **border-default** with **indigo-500** text.
 - *Tertiary/Ghost:* Inline. Transparent background, **indigo-500** text.

- *Destructive*: Red background for hazardous delete actions.
- **Input Focus State**: Text inputs must style focus outlines with `border-focus` (`indigo-500`) and a soft ring shadow `0 0 0 3px rgba(99, 102, 241, 0.1)` .
- **Card Padding & Borders**: Cards utilize `radius-lg` (12px) corners. Interactive cards transition using `transform: translateY(-2px)` on hover, with a border-color transition to `indigo-300` .
- **Badge Types**: Use subtle styling (light background with colored text) for normal indicators. Solid indigo badge gradients (`indigo-500` to `violet-600`) represent premium feature upgrades.

Edge Cases

- **Modal Overlay Isolation**: Form and alert modals must be displayed above a faded backdrop overlay, blocking page scroll underneath, trapping focus controls (`tab` key loops).
- **Mobile Table Adaptation**: On screens below `768px` , dense interactive tables must collapse into individual card grids or support swipe horizontal scrolling.

Technical Notes

- Toasts use a stack controller displaying a maximum of 3 notifications, auto-dismissing after 4s (success) or remaining until dismissed (errors).
- accessibility (ARIA) attributes like `role="dialog"` , `aria-modal="true"` , and `aria-live="polite"` must be wired natively.

Related Documents

- [Design System Overview](#)
- [Design Tokens](#)
- [Design Patterns](#)

 docs/design-system/patterns.md

Design System Patterns

Overview

Patterns document the standardized combinations of components to solve recurring UX layout problems (Forms, Navigation, Feedback, Visualizations).

Responsibilities

- Guide layout structures for uniform interactions across TakeSeat.
- Enforce correct layout alignments for sidebar, tabs, grids, and filters.
- Define standard feedback flows for loading (skeletons), empty states, and errors.

Architecture / Flow

Establish consistent layout templates:

1. **Forms:** Vertical stack, inline validation on blur, primary button on bottom right.
2. **Feedback States:** Action → show loading skeleton/spinner → success toast or error block → option to retry.
3. **Data Filtering:** Filter toolbar → triggers request → updates data table/chart layout.

Rules

- **Form Layout:** Fields are stacked vertically with labels placed on top. Required fields must use an indigo asterisk (*).
- **Tab Layout:** Active tabs get an indigo underline border (`border-bottom: 2px solid var(--indigo-500)`) and text color. Hovering inactive tabs displays

a subtle indigo background (`#EEF2FF`).

- **Sidebar Layout:** Fixed left panel. Collapsed state shows icons only with hover tooltips. Active links use `border-left: 3px solid var(--indigo-500)` with `bg-brand-subtle` .
- **Skeleton Shimmers:** Use a subtle indigo/gray gradient animation to represent content currently loading in grids or KPI charts.
- **Empty States:** Must include a large centered icon (with indigo tint), a clear title, descriptive helper text, and a primary brand button for recovery.

Edge Cases

- **Multi-step Forms:** Steps must auto-save progress, validate inputs before advancing, and allow users to go back without losing data.
- **Chart Palette:** Recharts visualization colors must map strictly to `indigo-500` (primary), `violet-500` (secondary), and `indigo-400` (tertiary) to ensure accessibility.

Technical Notes

- Form validation uses `react-hook-form` coupled with `zod` schemas.
- Dual-axis line charts (Wait times vs Volume) in Reports are responsive to screen size.

Related Documents

- [Design System Overview](#)
- [Design Tokens](#)
- [Design Components](#)

 docs/architecture/overview.md

System Architecture Overview

Overview

TakeSeat is a multi-tenant SaaS platform built to replace physical guest pagers and paper waitlists in hospitality. It routes guest actions, updates table queues, and notifies customers via WhatsApp.

Responsibilities

- **Multi-Tenant Isolation:** Scope all business data operations under logical partition identifiers.
- **Serverless Scaling:** Manage API scaling dynamically with zero idle server overhead.
- **Pluggable Messaging:** Abstract SMS and WhatsApp integrations to support easy provider changes.

Architecture / Flow

1. **Hostess UI:** SPA frontend loaded from CDN issues HTTPS REST requests.
2. **API Gateways:** AWS API Gateway forwards requests to AWS Lambda running Express.
3. **Domain Services:** Handlers query the MySQL database, execute state machines, and enqueue notifications.
4. **Outbound Messaging:** Service dispatches webhook payloads to Z-API for message delivery.

[Diagrama Mermaid — ver fonte original]

Rules

- **Logical Tenant Partitioning:** The database table structure uses `restaurantId` on all operational entities. Every read, write, update, and deletion must scope queries with this ID.
- **Stateless Execution:** All runtime data must persist in MySQL. Local server memory must not cache session variables.

Edge Cases

- **Database Connection Caps:** Aurora MySQL connections are pooled and limited to prevent server exhaustion during Lambda concurrency spikes.

Technical Notes

- Multi-tenancy is logical, utilizing a shared database instance with tenant index scopes.

Related Documents

- [Backend Architecture](#)
- [Frontend Architecture](#)
- [Database Schema](#)

 docs/architecture/components.md

System Components

Overview

TakeSeat is split into a client SPA Web Frontend, an Express API Backend running on AWS Lambda, a relational MySQL database, and pluggable messaging integration providers.

Responsibilities

- **Frontend SPA:** Deliver the operator interface for queue management, reports, and settings.
- **Backend API:** Expose authenticated endpoints, process business actions, and validate structures.
- **Database Storage:** Store records with relational integrity.
- **Migration Runner:** Perform database schema updates via CI/CD execution.

Architecture / Flow

- **Request Lifecycle:**
 1. Client SPA captures user actions → compiles payload → sends REST request.
 2. Backend Lambda Express middleware intercepts → validates token → executes controller → queries database.
 3. Service executes callbacks → triggers WhatsApp integrations.
- **Schema Management:**
 1. CI/CD initiates build → deploys `takeseat-migrate-prod` Lambda function.

2. Migration function runs `prisma migrate deploy` directly against the database inside the secure VPC.

Rules

- **No Direct DB Access from UI:** The React client must never contact the MySQL database directly. All operations must route through the secure REST API.
- **Domain Service Separation:** Core business rules (Waitlist, Customers, Auth) must remain isolated in dedicated Service classes inside the API.

Edge Cases

- **VPC Net Isolation:** Database instances are isolated inside private subnets. The API Lambda and Migration Lambda must reside inside the same VPC/Subnet route to resolve DB queries.

Technical Notes

- Technologies: React (Vite/TypeScript), Node.js (TypeScript/Express), MySQL (Prisma ORM), AWS Lambda, Z-API.

Related Documents

- [System Architecture Overview](#)
- [Database Schema](#)

 docs/backend/backend-architecture.md

Backend Architecture

Overview

The TakeSeat backend is designed as a modular Serverless Monolith. In production, it runs inside an AWS Lambda function handled by `@vendia/serverless-express`, exposing standard REST endpoints mapped via Amazon API Gateway. In local development, it starts a persistent Node.js HTTP server.

Responsibilities

- **Request Processing:** Host incoming REST API routes and map standard Express responses.
- **Tenant Context Scoping:** Dynamically associate request execution with the authenticated `restaurantId`.
- **Database Querying:** Retrieve and persist system records using Prisma ORM.
- **External Integration Abstraction:** Connect to WhatsApp (Z-API) and Stripe interfaces using modular providers.

Architecture / Flow

- **Execution Flow:**
 1. API Gateway routes request `/{{proxy+}}` to Lambda → `@vendia/serverless-express` converts AWS request schema to standard Express `req / res`.
 2. Middleware Layer parses headers, executes JWT token validation (`authenticate`), loads impersonation variables, and asserts subscription validity (`checkSubscriptionAccess`).

3. Router Layer delegates request to the associated Controller (e.g. `/waitlist` → `WaitlistController`).
4. Controller Layer validates input data structures using Zod schemas and executes the domain Service (e.g., `WaitlistService`).
5. Service Layer performs database operations via `PrismaClient` and triggers notifications before returning results.

Rules

- **Layered Architecture Isolation:** Controllers must not contain SQL queries or Prisma commands directly. All data mutations, business checks, and notification dispatches must reside in the Service layer.
- **Strict Tenant Boundaries:** All query operations in Services must explicitly match `where: { restaurantId }` . Under no circumstances should data be returned without this condition, preventing cross-tenant security breaches.
- **Provider Pattern Conformity:** All integrations with WhatsApp APIs must implement the `IWhatsAppProvider` interface. Direct references to API endpoints inside services are forbidden.

Edge Cases

- **AWS Lambda Execution Context Cold Starts:** DB connection limits must be managed carefully. Prisma connection pools must be configured to prevent exhausting MySQL connection limits when multiple Lambda instances scale up concurrently.
- **WebSocket Limits in Serverless:** Although `socket.io` websocket logic is defined, it is not used in the serverless environment because AWS Lambda is stateless and transient. Real-time updates rely instead on REST requests.

Technical Notes

- Runtime: Node.js 20.x, built using TypeScript compiling to standard CommonJS (`dist/server.js`).

- Database ORM: Prisma Client matching MySQL compatibilities.

Related Documents

- [API Patterns](#)
- [Database Schema](#)
- [Infrastructure Overview](#)

 docs/backend/api-patterns.md

API Patterns

Overview

This document outlines the design standards, request-response layouts, validation mechanisms, and error routing patterns for the TakeSeat RESTful API.

Responsibilities

- Define RESTful URL routing standards and HTTP verb mappings.
- Enforce strict input structures using schemas.
- Standardize the HTTP status codes and error payloads returned to clients.

Architecture / Flow

1. **Request Intake:** Client issues request (e.g. POST `/waitlist`) with payload.
2. **Schema Verification:** Router runs the associated validator middleware:

```
validateBody(createWaitlistEntrySchema)
```

If validation fails, the request is blocked, returning a `400 Bad Request` immediately.

3. **Execution:** If valid, the controller processes the payload and delegates to the service.

4. **Error Interception:** If an exception is thrown in the service, the global `errorHandler` middleware catches it, formats the message, and returns the appropriate HTTP code.

Rules

- **Authentication Header:** All private endpoints require the header `Authorization: Bearer <JWT_Token>`.
- **Response Format:** All successful responses must return a JSON payload with standard properties. Wrap list operations inside `data` envelopes accompanied by pagination metadata if applicable.
- **Input Validation:** All payloads (`req.body` , `req.query` , `req.params`) must be validated using **Zod** validator schemas. No raw manual properties evaluation is permitted in controllers.
- **HTTP Status Codes:**
 - `200 OK` : Successful read or update operation.
 - `201 Created` : Successful resource generation.
 - `400 Bad Request` : Input validation failed.
 - `401 Unauthorized` : Missing or invalid auth token.
 - `402 Payment Required` : Expired subscription or unpaid trial.
 - `403 Forbidden` : Authenticated user lacks permission scope for this action (RBAC).
 - `404 Not Found` : Resource does not exist.
 - `500 Internal Error` : System failure or database query error.

Edge Cases

- **Impersonation Requests:** When an admin impersonates a client, requests contain the header `X-Impersonate-Restaurant-Id`. The custom `impersonationMiddleware` overrides `req.user.restaurantId` to execute operations in the target tenant context safely.

- **Zod Error Detail Arrays:** When validation fails, the API returns a structured array of fields containing details of validation rules violated.

Technical Notes

- Validators reside in `backend/src/validators/` .
- Error handler middleware is configured globally in `backend/src/server.ts` .

Related Documents

- [Backend Architecture](#)
- [Security Overview](#)

 docs/frontend/frontend-architecture.md

Frontend Architecture

Overview

The TakeSeat frontend is a single-page React application built on Vite and TypeScript. It is optimized for mobile browser viewports, providing hostesses with operational tools to manage restaurant guest flows.

Responsibilities

- **App Routing & Guards:** Establish public, private, and subscription-protected routes.
- **Visual Presentation:** Render the UI using styled HTML/CSS following design tokens.
- **Localisation & Localization:** Handle translation switching across English, Portuguese (Brazil), and Spanish namespaces.
- **Client Networking:** Formulate API requests and sync local UI state via Axios and TanStack Query.

Architecture / Flow

- **Entrypoint:** `src/main.tsx` initializes React, the `LanguageContext`, the React Query Client, and renders `App.tsx`.
- **Global Contexts:**
 - `AuthContext` : Manages JWT access tokens, active tenant ID, and user details.
 - `ImpersonationContext` : Enables administrators to act on behalf of another restaurant tenant.

- `LanguageContext` : Synchronizes translation locales via `react-i18next` .
- **View Layer:** Components are divided into `components` (common UI widgets, inputs, and shell layouts) and `pages` (full views representing routes like `/waitlist` , `/reports` , `/settings`).

Rules

- **Directory Layout Structure:**
 - `src/components` : UI primitives (dialogs, tables, shells, layouts).
 - `src/contexts` : Global application contexts (auth, i18n, impersonation).
 - `src/hooks` : Global React hooks (auth, subscription limits).
 - `src/pages` : Individual page views.
 - `src/locales` : Localization files sorted as `{lang}/{namespace}.json` .
- **Language Detection:** The system uses `i18next-browser-languagedetector` to automatically determine language preference, syncing it with the backend preferred locale.
- **Routing Protection hierarchy:**
 - Public routes: `/auth` (Login/Register).
 - Authenticated private routes: `/onboarding` , `/verify-email` .
 - Subscription-gated routes: `/waitlist` , `/reports` , `/settings` .

Edge Cases

- **Chrome Mobile Viewport Resizing:** Native browser control bars hiding/showing on scroll can distort layouts. The application root locks the viewport height to `h-dvh` and wraps sub-views in a single-scroll `PageShell` container to prevent jitter.

- **Language Key Fallback:** If a localized text key is missing in Portuguese or Spanish, the i18n engine falls back to English.

Technical Notes

- Styling: Done primarily through custom Tailwind CSS utility rules.
- HTTP Requests: Configured with standard headers via an Axios interceptor that injects the active JWT `Authorization` header.

Related Documents

- [State Management](#)
- [Design System Overview](#)
- [Security Overview](#)

 docs/frontend/state-management.md

State Management

Overview

TakeSeat separates local UI states (form inputs, toggle flags) from server-side states (active waitlist entries, tenant profile data) using a combined strategy of React Context and TanStack (React) Query.

Responsibilities

- **Server Syncing:** Fetch, cache, and update server resources using declarative Query hooks.
- **Cache Invalidation:** Automatically trigger stale-state cache refetches when updates occur (mutations).
- **Global Contexts:** Manage small, stable, cross-component states (Authentication, Impersonation, Language).

Architecture / Flow

1. **Initial Mount:** Component runs a query hook (e.g., `useQuery({ queryKey: ['waitlist'] })`) → returns loading state and cache data.
2. **Action Dispatch:** User submits a form → triggers mutation hook (e.g., `useMutation`) → POST/PUT Axios request to backend.
3. **Invalidation:** On mutation success, the hook executes `queryClient.invalidateQueries({ queryKey: ['waitlist'] })`.

4. **Re-fetch:** React Query marks the cache as stale and issues background fetches, causing connected components to re-render.

Rules

- **Server State Gating:** All remote API endpoints must be managed via React Query. Do not keep local duplicate state variables of server datasets using `useState` hooks.
- **Key Conventions:** Maintain standardized query key arrays:
 - `['waitlist']` : List of active queue entries.
 - `['customers']` : Customer list and histories.
 - `['settings']` : WhatsApp templates and restaurant profiles.
 - `['users']` : Team accounts.
- **Mutations Side-Effects:** Always implement toast indicators (using `react-hot-toast`) on mutations (`onSuccess` , `onError`) to provide instant user feedback.

Edge Cases

- **Stale Cache Handling:** When a hostess runs the app on a mobile device and leaves it in the background, reactivation must trigger automatic focus refetches (`refetchOnWindowFocus: true`).
- **Websocket Real-Time Updates:** Although `socket.io-client` configuration exists in `src/lib/socket.ts` , it is currently not integrated to auto-invalidate React Query caches. The app relies on explicit user actions or query polling/refetches to sync.

Technical Notes

- Controlled in `frontend/src/lib/react-query` or `src/App.tsx` .
- Impersonation token storage is maintained in `localStorage` to persist administrator sessions across reloads.

Related Documents

- [Frontend Architecture](#)
- [Design System Patterns](#)

 docs/business-rules/queue-rules.md

Queue Rules (Waitlist)

Overview

The Waitlist is the core operational module of TakeSeat. It governs customer registration, status changes, waiting estimates (ETA), priority routing, and real-time status notifications.

Responsibilities

- **Registration:** Capture customer details, validation of active entries, and auto-calculating ETAs.
- **State Management:** Guide the transition of a queue entry from creation through to completion (**SEATED** , **CANCELLED** , or **NO_SHOW**).
- **Priority Rules:** Enforce priority queue ranking so prioritized customers bubble up.
- **Position Broadcasting:** Recalculate queue positions and trigger notifications for waiting parties when queue states shift.

Architecture / Flow

1. **Hostess Adds Entry:** Axios POST **/waitlist** → **WaitlistService.createEntry** → creates entry and updates customer details.
2. **Entry Lifecycle:**
 - **WAITING** : Initial state in queue.
 - **CALLED** : Hostess clicks "Call". Status moves to **CALLED** , triggers your-turn template.

- **SEATED** : Customer takes a seat. Updates customer profile with visits metrics.
 - **CANCELLED** : Customer leaves waitlist.
 - **NO_SHOW** : Called customer did not return.
3. **Trigger Updates:** Any terminal state update (**SEATED** , **CANCELLED** , **NO_SHOW**) triggers `notifyQueueUpdates()` asynchronously.

Rules

- **No Active Double-Booking:** A customer cannot be added to the queue if they have an existing entry with the status **WAITING** under the same restaurant.
 - *Exception:* If the customer's active entry is **CALLED** , the system allows inserting a new entry (assuming the prior position is being closed out or was missed).
- **Default Wait Time Formula:** If estimated minutes are not manually overridden by the hostess:
$$\text{ETA} = (\text{Total parties waiting in queue} + 1) \times 15 \text{ minutes}$$
- **Sorting Logic:** Waiting lists are fetched via:
 1. Priority flag descending (`isPriority: 'desc'`)
 2. Creation date ascending (`createdAt: 'asc'`) This means priority entries are always served first, in the order they entered.

Edge Cases

- **Plan Enforcement:** Only restaurants on the **PRO** plan can send WhatsApp welcome and position updates. If a restaurant is downgrading or on the **BASIC** plan, `effectiveWhatsappOptIn` is forced to `false` on entry creation.
- **Priority Persistence:** Activating/deactivating priority status on a waitlist entry immediately persists the priority preference back to the **Customer** record for future visits.

Technical Notes

- Metrics are processed in a rolling window of 90 minutes (`windowMinutes = 90`) comparing `calledAt` to `createdAt` to determine average wait time.
- Fallback average wait time when there is no historical data in the 90-minute window is 15 minutes.
- Real-time updates utilize standard HTTP polling and socket updates (future/partially implemented).

Related Documents

- [Customer Rules](#)
- [Subscription Rules](#)
- [WhatsApp Integration](#)

 docs/business-rules/customer-rules.md

Customer Rules

Overview

The Customer module handles persistent records of the visitors of each establishment. It serves as a light CRM that tracks visits, stores notes, persists priority preferences, and captures WhatsApp opt-in compliance histories.

Responsibilities

- **Deduplication:** Match new queue entries to existing customer records using clean phone identifiers.
- **Opt-In Persistence:** Manage customers' communication permissions with accurate audit source trails.
- **Visits History & Metrics:** Count successful seatings and log dates of their last visits.

Architecture / Flow

1. **Hostess Adds Entry:** The system runs `CustomersService.upsertCustomer` or matches details during `WaitlistService.createEntry`.
2. **Matching & Linking:**
 - If the cleaned `fullPhone` (DDI + Phone) already exists in the tenant's database, the customer profile is linked to the new entry, and name/notes/opt-in preferences are updated.
 - If not found, a new `Customer` record is generated.

3. **Visits Accumulation:** When a waitlist entry transitions to `SEATED`, the customer's `totalVisits` is incremented by 1, and `lastVisitAt` is set to the seating timestamp.

Rules

- **Cleaning Phone Identifiers:** All phone numbers must be sanitized before comparison, stripping spaces, dashes, brackets, and non-numeric characters. The country DDI is prepended with `+`.
- **Deduplication Key:** Customer uniqueness is strictly bounded by the composite key `(restaurantId, fullPhone)`. The same phone number can exist across different restaurants (tenants), but must be unique within one restaurant.
- **Opt-In Change Attribution:** When WhatsApp status changes, the change timestamp `(whatsappOptInAt)` is refreshed and the origin of modification is stored in `whatsappOptInSource`:
 - `CRM`: Modified through the Admin CRM dashboard.
 - `QUEUE_ENTRY`: Updated during the check-in queue registration.
 - `IMPORT`: Defaults to false on CSV imports unless specified.

Edge Cases

- **Opt-Out Precedence:** If a customer explicitly opts out of WhatsApp messages, subsequent waitlist entries default to opted-out. The hostess can toggle it back on if requested, which will overwrite the profile.
- **Email Fields:** Email is optional. If left blank, it is persisted as `null`.

Technical Notes

- Implemented in `CustomersService` and persisted in the `Customer` schema model.
- Deletions: Deleting a customer record clears their details but preserves independent waitlist logs (history of raw inputs).

Related Documents

- [Queue Rules](#)
- [Subscription Rules](#)
- [Database Schema](#)

 docs/business-rules/subscription-rules.md

Subscription Rules

Overview

The Subscription module defines the monetization rules, trial terms, payment gates, and access controls for the TakeSeat SaaS platform.

Responsibilities

- **Trial Lifecycle:** Handle the automatic trial setup for new business registrations.
- **Billing State Enforcement:** Maintain the active, past-due, and expired states based on billing webhooks.
- **Feature Gating:** Restrict backend middleware routes and frontend access based on subscription eligibility.

Architecture / Flow

1. **Restaurant Signup:** During registration, `SubscriptionService.startTrial` is invoked.
2. **Billing State Machine:**
 - Status moves to `TRIALING` (7 days duration).
 - Once trial ends without Stripe config: moves to `EXPIRED`.
 - Successful Stripe payment event: transitions to `ACTIVE`, sets `plan = 'PRO'`.
 - Failed Stripe payment event: transitions to `PAST_DUE` or `EXPIRED`.

3. **Middleware Gate:** `checkSubscriptionAccess` middleware evaluates each incoming route request against the tenant's current subscription status.

Rules

- **Access Eligibility:** Access to Waitlist, Reports, and Settings is only allowed if the restaurant's subscription status is `ACTIVE` or `TRIALING`.
- **Blocked State:** When status is `EXPIRED` or `PAST_DUE`, requests to protected resource endpoints return a `402 Payment Required` or redirect the client to `/billing`.
- **Trial Terms:** Trials run for exactly 7 days ($\$7 \times 24 \times 60 \times 60 \times 1000$ milliseconds).
- **Default Plan:** New restaurants default to the `PRO` plan during trial to test the WhatsApp notifications feature.

Edge Cases

- **Stripe Subscription Sync:** If a webhook event from Stripe fails or is delayed, the local status could fall out of sync. Standard database state reconciliation must handle Stripe webhook retries gracefully.
- **Past Due State:** A grace period status (`PAST_DUE`) is assigned when payment fails temporarily. The dashboard alerts the user but blocks queue check-ins immediately until payment succeeds.

Technical Notes

- Controlled in `SubscriptionService` and applied globally via routes authentication and the `checkSubscriptionAccess` middleware.
- The `plan` enum field contains `BASIC` and `PRO`. The `BASIC` plan remains in the schema for backwards compatibility but is no longer actively marketed or selectable.

Related Documents

- [Features List](#)
- [Stripe Integration](#)
- [Security Overview](#)

 docs/database/schema-overview.md

Database Schema Overview

Overview

TakeSeat uses a MySQL relational database managed via Prisma ORM. The schema is optimized for multi-tenant isolation, high-speed waitlist queries, audit logging, and customer record deduplication.

Responsibilities

- **Multi-Tenant Partitioning:** Ensure all operational entities map back to a specific `Restaurant` parent.
- **Deduplication Safeguards:** Apply database-level unique keys to prevent duplicate active users.
- **Operational Auditing:** Retain records of message logs and administrator impersonation actions.

Architecture / Flow

[Diagrama Mermaid — ver fonte original]

Rules

- **Foreign Key Cascade Deletions:** Deleting a `Restaurant` cascade-deletes its associated `User`, `Customer`, `WaitlistEntry`, `RestaurantWhatsAppSettings`, and audit records.
- **Compound Unique Keys:** The `Customer` table has a composite unique constraint `@@unique([restaurantId, fullPhone])` to ensure unique

customer identity profiles per tenant.

- **Database Migrations:** All schema alterations must be handled through declarative SQL migration scripts within Prisma (`prisma migrate dev`). Direct database modifications are strictly prohibited.

Edge Cases

- **Customer Nullification on Queue Deletion:** If a `Customer` is manually deleted from the CRM, connected `WaitlistEntry` records are not deleted; instead, their `customerId` foreign key is set to `null` (`onDelete: SetNull`) to preserve anonymous metrics.

Technical Notes

- Indexing is heavily applied to `WaitlistEntry` on composite keys `(restaurantId, status, createdAt)` and `(restaurantId, seatedAt)` to optimize the performance of waitlist dashboard loading and reports generation queries.

Related Documents

- [Backend Architecture](#)
- [Subscription Rules](#)
- [Setup Guide](#)

 docs/database/entities/restaurants.md

Restaurants Entity

Overview

The **Restaurant** model represents the primary tenant in TakeSeat. All other entities (except global SYSADMIN users) belong directly to a restaurant.

Responsibilities

- Hold restaurant business metadata (CNPJ, Trade Name).
- Maintain localized settings (Timezone, ISO Country Code, Address).
- Act as the root anchor for cascading data deletions.

Architecture / Flow

- Mapped to the **restaurants** table in MySQL.
- Holds one-to-many relationships with **User** , **Customer** , **WaitlistEntry** , and **WhatsAppMessageLog** .
- Holds a one-to-one relationship with **RestaurantWhatsAppSettings** .

Rules

- **Required Fields:** **name** , **city** , **timezone** , and **isActive** are mandatory.
- **Default country code:** Defaults to **"BR"** .
- **Default timezone:** Defaults to **"America/Sao_Paulo"** .

Edge Cases

- **Onboarding Wizard State:** The `onboardingPending` flag defaults to `true` on creation. Until it is flipped to `false` via onboarding completion, dashboard pages are locked.

Technical Notes

- timezone string matches standard IANA database locations (e.g. `America/Sao_Paulo`).

Related Documents

- [Database Schema Overview](#)
- [Subscription Entity](#)

 docs/database/entities/customers.md

Customers Entity

Overview

The **Customer** model represents a persistent CRM record of a guest who has visited or joined the waitlist of a tenant restaurant.

Responsibilities

- Store guest identifiers (name, email, notes).
- Hold cleaned international phone attributes (**countryCode** , **ddi** , **phone** , **fullPhone**).
- Track CRM loyalty metrics (**totalVisits** , **lastVisitAt**).
- Persist WhatsApp communication opt-in histories.

Architecture / Flow

- Mapped to the **customers** table in MySQL.
- Belongs to a **Restaurant** .
- Has a one-to-many relationship with **WaitlistEntry** .

Rules

- **Clean Phone Constraint:** Phone attributes are parsed into individual country codes, DDI, and localized numbers.

- **Uniqueness Boundary:** Unique combination of `(restaurantId, fullPhone)` ensures a guest cannot have duplicate profiles under the same brand tenant.

Edge Cases

- **No-Show and Cancellation Exclusions:** Seating a guest increments `totalVisits` and sets `lastVisitAt` . However, when a queue entry is marked as `NO_SHOW` or `CANCELLED` , these loyalty metrics are untouched.

Technical Notes

- Indexing: Added on `[restaurantId]` , `[fullPhone]` , and `[email]` for query optimization in dashboard filters.

Related Documents

- [Database Schema Overview](#)
- [Customer Rules](#)

 docs/database/entities/waitlist_entries.md

Waitlist Entries Entity

Overview

The `WaitlistEntry` model logs each individual check-in occurrence of a customer joining a restaurant's queue.

Responsibilities

- Record immediate operational variables (party size, priority status, estimated wait minutes).
- Transition waitlist states (`WAITING` , `CALLED` , `SEATED` , `CANCELLED` , `NO_SHOW`).
- Retain lifecycle action timestamps (`createdAt` , `calledAt` , `seatedAt` , `cancelledAt` , `noShowAt`).

Architecture / Flow

- Mapped to the `waitlist_entries` table in MySQL.
- Belongs to a `Restaurant` and optionally links to a `Customer` (via `customerId`).

Rules

- **Status Lifecycle Transitions:**
 - Starts as `WAITING` .

- Can move to **CALLED** (when hostess notifies that table is ready).
- Can transition to **SEATED** (terminal success), **CANCELLED** (terminal customer action), or **NO_SHOW** (terminal staff action).
- **Opt-In Sync:** Mirrors customer's opt-in choice during registration.

Edge Cases

- **Customer Deletion Resilience:** If a customer record is deleted, **customerId** is set to **null** to retain anonymous metrics for queue duration analytics, instead of deleting the entry record.

Technical Notes

- Indexing: Mapped composite indexes **[restaurantId, status, createdAt]**, **[restaurantId, seatedAt]**, and **[restaurantId, calledAt]** to enable real-time dashboard renders and queue analytics metrics calculations without table scans.

Related Documents

- [Database Schema Overview](#)
- [Queue Rules](#)

 docs/database/entities/subscriptions.md

Subscriptions Entity

Overview

Subscription attributes are embedded directly in the **Restaurant** model, mapping plan classifications and trial parameters without requiring joins.

Responsibilities

- Track active tenant billing levels (**plan**).
- Maintain billing state records (**subscriptionStatus**).
- Log date indicators of billing state transitions.

Architecture / Flow

- Mapped as core columns inside the **restaurants** table.
- Monitored by middleware gates before serving resources.

Rules

- **Plan Tiers:**
 - **PRO** : Full access. WhatsApp notifications active.
 - **BASIC** : Legacy plan. Restricted access.
- **Subscription Statuses:**
 - **TRIALING** : Temporary trial.
 - **ACTIVE** : Normal paid subscription.

- `PAST_DUE` : Payment failed, grace period.
- `EXPIRED` : Access suspended.

Edge Cases

- **Trial Fields Backward Compatibility:** Legacy fields `trialStatus` , `trialStartAt` , `trialEndAt` , and `trialConsumedAt` are retained in the schema for backward compatibility with earlier migrations, but runtime state evaluations check `subscriptionStatus === 'TRIALING'` .

Technical Notes

- Webhook endpoints directly modify these columns upon receiving validated Stripe signature payloads.

Related Documents

- [Database Schema Overview](#)
- [Subscription Rules](#)

 docs/database/entities/users.md

Users Entity

Overview

The **User** model represents administrative, management, and host staff accounts that log in to manage restaurant operations.

Responsibilities

- Hold authentication identifiers (email, hashed password).
- Restrict interface permissions through RBAC (Role-Based Access Control) assignments.
- Store email verification tokens and status.

Architecture / Flow

- Mapped to the **users** table in MySQL.
- Belongs to **Restaurant** (optional relationship, since platform-wide **SYSADMIN** users do not link to any single restaurant).

Rules

- **Unique Fields:** The **email** field has a unique database constraint.
- **Roles:** Must belong to the **UserRole** enum:
 - **SYSADMIN** : Platform-wide admin with impersonation rights.
 - **ADMIN** : Restaurant account owner.

- **MANAGER** : Store manager with settings control.
- **HOSTESS** : Operational staff restricted to waitlist activities.

Edge Cases

- **Nullable Tenant Relationship:** **restaurantId** is nullable. When **null**, the system evaluates the user role; if not **SYSADMIN**, access is denied.

Technical Notes

- Password hashes are stored using bcrypt algorithms.
- Indexes: Applied on **[restaurantId]** and **[email]** for rapid login lookup.

Related Documents

- [Database Schema Overview](#)
- [Security Overview](#)

 docs/flows/authentication-flow.md

Authentication Flow

Overview

This document describes the user access flows for administrative staff logging into the dashboard and guest customers accessing public tracking pages.

Responsibilities

- Issue secure JSON Web Tokens (JWT) for authenticated operator sessions.
- Authorize read-only guest tracking views using obfuscated endpoint tokens.

Architecture / Flow

- **Staff Login Flow:**
 1. Operator submits email and password via `POST /auth/login`.
 2. Backend looks up email, evaluates bcrypt password hash, and checks if user profile is active.
 3. Backend generates short-lived JWT Access Token and long-lived Refresh Token.
 4. Frontend stores tokens, intercepts Axios requests to inject authorization headers, and initializes the layout language matching `user.language`.
- **Guest Tracking Link Flow:**
 1. Guest navigates to link delivered in WhatsApp notification (e.g., `/queue/abcd-1234`).
 2. UI requests page info using `GET /queue/:publicToken`.

3. Backend parses token, checks database for associated `WaitlistEntry` , and returns wait metrics if valid.

Rules

- **Access Privilege Limits:** Guests do not log in. Their access token allows read-only visibility for their specific waitlist record. Guests cannot query other queue entries or mutate queue statuses.

Edge Cases

- **Revoking Active Users:** When a manager toggles a user to inactive (`isActive = false`), the session JWT remains valid until expiration. Future refresh token requests are immediately rejected.

Technical Notes

- Signature algorithm: HMAC-SHA256.

Related Documents

- [Security Overview](#)
- [API Patterns](#)

 docs/flows/billing-and-trial-flow.md

Billing and Trial Flow

Overview

TakeSeat operates a subscription SaaS model. Restaurants register under a 7-day free trial, transitioning later into active paid subscriptions or suspended states based on invoice payment events.

Responsibilities

- Track trial period durations and trigger expiration.
- Handle checkout creation and process payment outcomes.

Architecture / Flow

[Diagrama Mermaid — ver fonte original]

Rules

- **Onboarding Start:** New tenants default to plan **PRO** and status **TRIALING** for exactly 7 days.
- **Access Restrictions:** Features are locked when status is **EXPIRED** or **PAST_DUE**. The dashboard routes requests to **/billing**.
- **Payment Reconciliation:** Subscription activations and payment failures are handled asynchronously via verified Stripe Webhooks.

Edge Cases

- **Trial Expired Job:** A backend scheduled automation monitors date boundaries and updates expired `TRIALING` tenants to `EXPIRED` .

Technical Notes

- Subscriptions are checked via the `canAccessFeatures` method on `SubscriptionService` .

Related Documents

- [Subscription Rules](#)
- [Stripe Integration](#)

 docs/flows/queue-flow.md

Queue Flow

Overview

This flow maps the lifecycle of a guest party from joining the waitlist, checking position updates, being called, and being seated or marked as no-show/cancelled.

Responsibilities

- Track queue state transitions (**WAITING** → **CALLED** → **SEATED** / **NO_SHOW** / **CANCELLED**).
- Verify guest data structures and prevent duplicate active waitlist entries.
- Dispatch notifications on transition triggers.

Architecture / Flow

1. **Join Queue** (**POST /queue**): Hostess registers guest details. The system matches/creates the customer profile, saves **WaitlistEntry** as **WAITING** , and dispatches the Welcome notification.
2. **Track Status** (**GET /queue/:publicToken**): Guest tracks position, estimated wait minutes, and status updates via their public mobile page link.
3. **Call Party** (**POST /queue/:id/call**): Hostess triggers the call. Status moves to **CALLED** , initiating a visual return timer on the dashboard and dispatching the WhatsApp notification.
4. **Completion**:
 - Seating: Call **POST /queue/:id/seat** . Status changes to **SEATED** , incrementing customer loyalty stats.

- Cancel: Call `POST /queue/:id/cancel` . Status changes to `CANCELLED` .
- No-Show: Call `POST /queue/:id/noshow` . Status changes to `NO_SHOW` .

Rules

- **Double Entry Prevention:** A customer cannot be added to the waitlist if they have an active `WAITING` or `CALLED` queue record.
- **Loyalty Accrual:** Seating a guest is the only action that increments the customer's `totalVisits` and updates `lastVisitAt` .

Edge Cases

- **Priority Override:** Priority guests are sorted at the top of the queue (`isPriority desc, createdAt asc`), adjusting positions and estimated times for other parties.

Technical Notes

- Implemented in `WaitlistService` .

Related Documents

- [Queue Rules](#)
- [Customer Rules](#)

 docs/flows/reporting-flow.md

Reporting Flow

Overview

The Reporting flow aggregates historical waitlist data into metrics and visualizations to help operators analyze queue throughput and performance.

Responsibilities

- Aggregate ended waitlist entries over date ranges.
- Compute average wait times, called-to-seated durations, and cancellation percentages.
- Export raw tabular data.

Architecture / Flow

1. **Query Dispatch:** UI sends request `GET /reports/waitlist-performance?startDate=...&endDate=...` .
2. **Data Aggregation:** Backend queries database, filtering completed entries (`SEATED` , `CANCELLED` , `NO_SHOW`) by date range.
3. **KPI Computation:**
 - **Average Wait Time:** Difference between `seatedAt` / `calledAt` and `createdAt` .
 - **Abandonment Rate:** Percentage of cancelled/no-show entries vs total entries.
 - **Daily Peak Hours:** Grouping check-in records by hour of day.

4. **Data Delivery:** UI renders values on KPI cards and plots trend lines using Recharts components.

Rules

- **Date Range Limit:** The query date range must not exceed a maximum window of 90 days to prevent excessive database loading.
- **Tenant Scope isolation:** Aggregations must enforce `where: { restaurantId }` to ensure data privacy.

Edge Cases

- **No Data Fallback:** If zero entries exist in the selected range, the endpoint returns default zeroed metrics to prevent divide-by-zero runtime exceptions.

Technical Notes

- Implemented in `ReportsController` and displayed on the `/reports/waitlist-performance` route.

Related Documents

- [Features List](#)
- [Database Schema](#)

 docs/flows/user-management-flow.md

User Management Flow

Overview

This flow governs administrative operations to create staff accounts, update roles, and manage credentials within a tenant restaurant.

Responsibilities

- Create and update user accounts.
- Restrict access privileges using roles.
- Manage password reset credentials.

Architecture / Flow

- **Create User Flow:**
 1. Admin clicks "Add Team Member" → fills name, email, password, and role.
 2. Router verifies creator has role `ADMIN`.
 3. Backend checks if email is unique → hashes password → persists record associated with creator's `restaurantId`.
 4. Language is set to default language of creator user profile.
- **Update Status Flow:**
 - Admin can update user status to inactive (`isActive = false`), immediately restricting future authentication operations.

Rules

- **Role Hierarchy Controls:**
 - Only `ADMIN` role users can add, edit, or disable team members.
 - `MANAGER` and `HOSTESS` roles are forbidden from accessing team management routes.

Edge Cases

- **Self-Disabling Guard:** Admins are blocked from disabling their own accounts or changing their own role to prevent lockout events.

Technical Notes

- Controlled in `users-management.routes.ts` and managed in `UsersManagement.tsx`.

Related Documents

- [Features List](#)
- [Security Overview](#)

 docs/flows/whatsapp-flow.md

WhatsApp Notification Flow

Overview

This flow governs how guest notifications are generated, compiled, dispatched, and audited across waitlist transition triggers.

Responsibilities

- Compile message templates with real-time variables.
- Manage opt-in validation checks.
- Audit outbound notifications in database logs.

Architecture / Flow

1. **Trigger Event:** A guest action triggers a state change (e.g. hostess calls party → status becomes `CALLED`).
2. **Opt-in & Settings Check:**
 - Check if guest `whatsappOptIn` is true.
 - Check if restaurant settings has enabled the specific notification.
3. **Template Interpolation:** Load text template from the database, substituting variables:
 - `{{customer_name}}` , `{{business_name}}` ,
`{{position}}` , `{{eta_minutes}}` .
4. **Provider Dispatch:** Dispatch payload containing recipient number and compiled message text to `WhatsAppProvider.sendText` .

5. **Log Generation:** Persist notification status and reference codes in

`WhatsAppMessageLog` .

Rules

- **No SMS Failover:** The application only uses WhatsApp notifications. If message dispatch fails, the system does not fallback to SMS.

Edge Cases

- **Z-API Webhook Status Feeds:** If the recipient's phone is disconnected, the provider returns delivery failure payloads to the system webhook, updating `WhatsAppMessageLog.status` to `FAILED` .

Technical Notes

- Controlled in `WhatsAppService` and executed asynchronously to prevent blocking API responses.

Related Documents

- [WhatsApp Integration](#)
- [Queue Rules](#)

 docs/integrations/whatsapp.md

WhatsApp Integration

Overview

TakeSeat integrates with WhatsApp to notify queue guests of their registration, position movements, and table call actions. Communication is handled through a unified client provider abstraction interfacing with the Z-API gateway.

Responsibilities

- **Outbound Dispatch:** Convert text notifications into JSON payloads and post them to Z-API endpoints.
- **Payload Normalization:** Sanitize phone numbers to comply with digit-only format requirements.
- **Rate-Limiting Controls:** Prevent spamming guests with redundant queue position update messages.

Architecture / Flow

1. **Trigger Event:** A guest action changes waitlist status (e.g. hostess calls party → `WaitlistService.callEntry`).
2. **Settings Audit:** Service checks `RestaurantWhatsAppSettings` :
 - Verify `isEnabled` is true.
 - Verify specific message types are enabled (`sendWelcome` , `sendPositionUpdates` , `sendTurnMessage`).
3. **Template Compilation:** Load text template from the database, substituting variables:

- `{{customer_name}}` , `{{business_name}}` , `{{position}}` , `{{eta_minutes}}` .

4. **Provider Dispatch:** The service invokes the singleton provider `WhatsAppProvider.sendText(options)` which forwards to Z-API.
5. **Logs Generation:** Persist an entry in `WhatsAppMessageLog` containing the response message ID or failure codes.

Rules

- **Environment Keys:** The integration relies on:
 - `ZAPI_BASE_URL` : API gateway endpoint.
 - `ZAPI_INSTANCE_ID` , `ZAPI_INSTANCE_TOKEN` , `ZAPI_CLIENT_TOKEN` : Authentication credentials.
- **Provider Isolation:** All service files must interact exclusively with the `IWhatsAppProvider` interface wrapper. Do not instantiate direct HTTP calls to Z-API outside the provider.
- **Opt-In Check:** The message handler must confirm `whatsappOptIn` is true on the specific waitlist entry before triggering the provider dispatch.

Edge Cases

- **Duplicate Welcomes on Re-entry:** If a guest is removed and re-added to the waitlist on the same day, they will receive a new Welcome notification.
- **Position Update Limits:** Position messages are only dispatched if:
 - Position changes by at least `minPositionsChangeToNotify` (default 5).
 - Time elapsed since last notification exceeds `minSecondsBetweenUpdates` (default 5 minutes).

Technical Notes

- The default provider is `ZApiWhatsAppProvider` , which strips non-numeric characters from the destination number prior to POST requests.

Related Documents

- [Queue Rules](#)
- [Customer Rules](#)

 docs/integrations/stripe.md

Stripe Integration

Overview

TakeSeat handles automated billing subscriptions by connecting to the Stripe payment API. The flow handles initial checkout redirections and registers webhook events to synchronize payment outcomes with local database subscription statuses.

Responsibilities

- **Checkout Initialisation:** Generate Stripe Checkout Sessions to capture payment methods.
- **Webhook Processing:** Intercept asynchronous Stripe billing events and reconcile customer states.
- **Metadata Management:** Embed reference tenant IDs in Stripe schemas to link transactions back to the correct restaurants.

Architecture / Flow

1. **Checkout Event:** User navigates to billing panel → clicks "Subscribe" → triggers `POST /billing/subscribe`.
2. **Redirect Flow:**
 - *Production:* Backend creates a checkout session at Stripe, returning the gateway checkout URL.
 - *Sandbox:* A placeholder redirection is returned redirecting back to `/billing?stripe-pending=true`.
3. **Webhook Callback:** Stripe issues a `POST /billing/webhook` containing the transaction result.

4. Reconciliation:

- `checkout.session.completed` → Call `subscriptionService.activateSubscription` .
- `invoice.payment_succeeded` → Call `subscriptionService.activateSubscription` (restoring service if past due).
- `invoice.payment_failed` → Call `subscriptionService.setPastDue` .
- `customer.subscription.deleted` → Call `subscriptionService.expireSubscription` .

Rules

- **Signature Security:** Every production webhook handler call must verify Stripe signatures using the endpoint raw payload and `STRIPE_WEBHOOK_SECRET` before processing the body.
- **Tenant Linking:** Stripe metadata fields must always include the `restaurantId` parameter. Stripe payloads lacking this property must be ignored and logged as errors.

Edge Cases

- **Missing webhook events:** When network failure stops a webhook from reaching TakeSeat, cron schedules or active dashboard checks must execute fallbacks calling the Stripe subscription lookup API.

Technical Notes

- Webhook endpoints route requests without calling Express authentication middleware.
- Environment keys: `STRIPE_SECRET_KEY` , `STRIPE_WEBHOOK_SECRET` , `STRIPE_PRO_PRICE_ID` .

Related Documents

- [Subscription Rules](#)
- [Database Schema](#)

 docs/decisions/ADR-001-whatsapp-provider.md

ADR 001: WhatsApp Provider Pattern

Overview

This document records the architectural decision to decouple outbound messaging integrations from specific providers using a Strategy/Provider pattern interface.

Responsibilities

- Define a uniform service interface for dispatching text alerts.
- Support swappable message transport implementations (Z-API, Meta Cloud API) without impacting business logic.

Architecture / Flow

`WaitlistService` → Calls interface method
`IWhatsAppProvider.sendText` → Invokes active injected subclass
(`ZApiWhatsAppProvider`) → Issues API request.

Rules

- All business services must interact solely with the `IWhatsAppProvider` interface abstraction.
- Subclass implementations must handle vendor-specific request layouts and response parsing internally.

Edge Cases

- **Development Mocking:** In non-production environments, a `MockWhatsAppProvider` can be resolved to log dispatches in terminal consoles instead of issuing real billing requests.

Technical Notes

- Current active subclass in production: `ZApiWhatsAppProvider` .

Related Documents

- [WhatsApp Integration](#)
- [System Components](#)

 docs/decisions/ADR-002-i18n-strategy.md

ADR 002: Internationalization Strategy

Overview

This document outlines the hierarchy used to resolve locales (English, Portuguese, Spanish) across admin views and guest-facing status screens.

Responsibilities

- Select the correct language preference for staff operators and guest clients.
- Provide a modular translation structure leveraging JSON dictionary files.

Architecture / Flow

1. **LoggedIn Operator:** Resolve locale from the `User.language` DB column.
2. **Guest public screen:**
 - Evaluate the `?lang=` URL parameter.
 - Fall back to the host restaurant's default settings language.
 - Fall back to the browser's language (`navigator.language`).

Rules

- Outbound WhatsApp notifications sent to guests must use the language configured by the restaurant profile, regardless of the active hostess interface locale.

Edge Cases

- **Missing Translations:** When a translation key is missing in the Portuguese or Spanish dictionaries, the i18n manager must fallback gracefully to the English key definition.

Technical Notes

- Implemented in React using `react-i18next` and backend query intercepts.

Related Documents

- [Frontend Architecture](#)
- [Features List](#)

 docs/decisions/ADR-003-multi-tenant-model.md

ADR 003: Multi-Tenant Database Design

Overview

This document records the selection of a shared database with logical discriminator columns to partition tenant data across the SaaS system.

Responsibilities

- Provide cost-effective resource utilization.
- Secure tenant data isolation through strict application-level scoping.

Architecture / Flow

- **Data Partitioning:** All tables containing customer, user, or waitlist metrics implement a mandatory `restaurantId` column referencing the parent restaurant.

Rules

- All application queries accessing tenant tables must verify that `where: { restaurantId }` filters are appended.

Edge Cases

- **Shared Analytics Leak Risk:** Security tests must regularly review APIs to ensure data is not exposed between tenants due to a missing filter constraint.

Technical Notes

- Implemented using Aurora Serverless v2 MySQL and Prisma Client code bindings.

Related Documents

- [Database Schema Overview](#)
- [Backend Architecture](#)

 docs/security/security-overview.md

Security Overview

Overview

This document outlines the security parameters, access controls, transport protocols, and data isolation strategies implemented across the TakeSeat application platform.

Responsibilities

- **Access Verification:** Validate identity credentials of platform operators and verify signatures.
- **Data Isolation Enforcement:** Guard tenant datasets from cross-tenant visibility leaks.
- **Secrets Management:** Prevent sensitive credentials and keys from exposure in static source files.

Architecture / Flow

- **Staff Access Flow:** Operator sends credentials → API validates password hash and issues signed JWT → client stores JWT → subsequent requests send JWT in **Authorization** header → middleware decrypts and checks role clearance.
- **Guest Access Flow:** Guest checks in → database generates UUID for **WaitlistEntry** → system generates public URL containing entry UUID → guest requests status via URL token (no password login required).

Rules

- **Password Hashing:** Store all passwords using bcrypt hashing. Plaintext storage is forbidden.
- **JWT Signing:** Access tokens are signed using the `JWT_SECRET` key via the HS256 algorithm.
- **Tenant Scope Enforcement:** All data service endpoints must filter database queries using `where: { restaurantId }`.
- **API CORS:** Whitelist only authorized domains (`takeseat.me` , local development origins) in Express CORS configurations.
- **Transport Layer Security:** HTTPS is mandatory across all channels (Vite UI via CloudFront, API routes via AWS API Gateway).

Edge Cases

- **Sysadmin Impersonation Security Audit:** When a system administrator impersonates a restaurant tenant, the actions are audited in `ImpersonationLog` records to log the operator ID, target restaurant, start/end timestamps, and reason.

Technical Notes

- **Input validation:** Implemented using Zod validation schemas to protect endpoints against injection attacks.

Related Documents

- [Database Schema Overview](#)
- [API Patterns](#)

 docs/ai-context/agent-guidelines.md

Agent Guidelines

Overview

This document outlines critical software invariants, architectural boundaries, and safety constraints that AI coding agents must respect when modifying the TakeSeat codebase.

Responsibilities

- Prevent regressions in security, authentication, and multi-tenant scoping.
- Protect the mobile responsive layout viewport configuration.
- Enforce clean phone number sanitization rules.

Architecture / Flow

1. **Understand Constraints:** Review repository boundaries and existing conventions before code edits.
2. **Execute Changes:** Modify logic using clean separation patterns.
3. **Verify Compliance:** Compile typescript, run linting checks, and execute local builds.
4. **Deploy Safely:** Push code cleanly through version control pipelines.

Rules

- **Multi-Tenant Safeguard:** Never omit the `restaurantId` filter in queries. Every database operation must restrict search queries to the active tenant's context.
- **Mobile Viewport Locks:** Do not alter the mobile viewport configuration. The root layout locks height using `h-dvh` and `overflow-hidden`, using

`overflow-y-auto` on the inner `PageShell` scroll wrapper to prevent address bar resizing bugs in mobile browsers.

- **Strict Billing Gates:** Do not bypass subscription middleware authorization routines. Only allow access to active or trialing tenants.
- **No Destructive Database Changes:** Do not mutate existing database schemas or delete Prisma columns without prior confirmation.

Edge Cases

- **Bcrypt Native Modules:** If compilation errors crop up in backend containers, ensure `npm rebuild bcrypt --build-from-source` is run to compile binaries correctly.
- **WebSocket Placeholders:** Do not hook or import WebSocket clients in the core UI context as AWS Lambda runtime states do not actively support persistent connections.

Technical Notes

- Build verification: Run `npm run build` inside both `frontend` and `backend` subdirectories to check for compilation issues.

Related Documents

- [System Architecture Overview](#)
- [Security Overview](#)

 docs/ai-context/system-context.md

System Context

Overview

This document defines the boundaries of the TakeSeat waitlist platform, listing user actors, external integrations, and core system boundaries.

Responsibilities

- Define actors (Guest Customers, Host Operators, Restaurant Admins).
- Establish communication boundaries with messaging gateways (Z-API).
- Delineate feature inclusions from system exclusions.

Architecture / Flow

[Diagrama Mermaid — ver fonte original]

Rules

- **Authentication Border:** Guests do not log in. Guest access is authorized exclusively via a secure obfuscated hash parameter within their waitlist tracking link.
- **Provider Decoupling:** All messaging requests route through Z-API interfaces. The core system remains platform-agnostic to facilitate transitioning to alternative providers (Meta Cloud API).

Edge Cases

- **Unverified Phone Delivery:** Z-API requires a valid destination number digits sequence. Delivery to invalid numbers will fail silently or return error logs to the webhook listener.

Technical Notes

- Integrates with Z-API endpoints using credentials stored in system variables.

Related Documents

- [System Architecture Overview](#)
- [WhatsApp Integration](#)

 docs/ai-context/domain-glossary.md

Domain Glossary

Overview

This glossary translates business, design, and technical definitions utilized across the TakeSeat codebase to align understanding between human developers and AI agents.

Responsibilities

- Define terminology representing core database entities.
- Standardize calculations and acronym abbreviations.
- Document third-party integration service names.

Architecture / Flow

Business concept (e.g. Fila) → Database representation (e.g. WaitlistEntry) → UI Presentation (e.g. Waitlist Card) .

Rules

- **Restaurant (Tenant):** The paying business entity hosting a queue. Mapped in code as **Restaurant** or **tenant** .
- **Waitlist (Fila de Espera):** The active queue of guest parties waiting for a table.
- **Customer (Cliente):** Persistent CRM profile, uniquely identified by country code + DDI + local phone number.
- **Waitlist Entry (Inscrição na Fila):** A transient log mapping a customer's specific check-in event.
- **DDI:** Direct Distance Inward (country phone prefix, e.g. **+55** for Brazil).

- **Z-API:** Non-official WhatsApp gateway routing HTTP webhook dispatches to target devices.
- **Impersonation (Simulação/Impersonificação):** administrative action allowing platform admins to manage a restaurant's dashboard without requesting their passwords.
- **Onboarding Wizard:** The initial setup form that completes missing restaurant variables (city, timezone) before unlocking the dashboard.
- **ETA (Estimated Time of Arrival / Tempo Estimado):** Wait calculation computed as $\$(N+1) \times 15\$$ minutes, where $\$N\$$ is the number of active waiting parties in the queue.

Edge Cases

- **Customer vs WaitlistEntry:** A `Customer` record is permanent. A `WaitlistEntry` is temporary and gets deleted or archived on completing terminal states.

Technical Notes

- Term translations are managed in standard JSON files inside `frontend/src/locales/`.

Related Documents

- [Features List](#)
- [Queue Rules](#)

 docs/setup/local-setup.md

Local Setup Guide

Overview

This document guides developers through setting up the TakeSeat development environment locally, including database provisioning, dependency installation, and running development servers.

Responsibilities

- Configure local environment configurations for backend and frontend.
- Launch development databases and run migrations.
- Verify running instances of local servers.

Architecture / Flow

1. **Prerequisites:** Install Node.js 20.x, npm, and Docker.
2. **Database Provisioning:** Start local MySQL instance.
3. **Backend Configuration:** Set up environment variables, run Prisma migrations, and start server.
4. **Frontend Configuration:** Configure variables and run Vite dev server.

Rules

- **Environment Isolation:** Never edit or commit `.env` secret variables into git repositories. Copy `.env.example` configurations to create local `.env` files.
- **Migration Sync:** Do not make manual changes to database schemas without creating a Prisma migration:

```
npx prisma migrate dev --name <migration_name>
```

Edge Cases

- **Bcrypt Native Module Compilation Failures:** If installing dependencies on macOS or Windows throws native C++ build errors, clean node modules and execute:

```
npm rebuild bcrypt --build-from-source
```

Technical Notes

- **Backend Port:** `http://localhost:3000`
- **Frontend Port:** `http://localhost:5173`
- **Recommended Database Compose:**

```
version: '3.8'
services:
  db:
    image: mysql:8.0
    command: --default-authentication-
plugin=mysql_native_password
    restart: always
    environment:
      MYSQL_DATABASE: takeseat_dev
      MYSQL_ROOT_PASSWORD: root
    ports:
      - '3306:3306'
```

Related Documents

- [System Architecture Overview](#)
- [Deployment Guide](#)

 docs/guides/email-service-runbook.md

Email Service Runbook

Overview

This runbook provides operational procedures for managing AWS SES (Simple Email Service) for the TakeSeat application. SES is used to send verification emails during the signup process.

Responsibilities

- Manage and monitor AWS SES v2 credentials and region identities.
- Verify DKIM, SPF, and DMARC record status to prevent message delivery blockages.
- Request production access to transition out of sandbox sending limits.

Architecture / Flow

- **Service:** AWS SES v2 (Simple Email Service)
- **Region:** `us-east-1` (configurable via `SES_REGION`)
- **From Address:** `contato@takeseat.me` (configurable via `SES_FROM_EMAIL`)
- **Identity Type:** Domain-based (`takeseat.me`)
- **Authentication:** DKIM, SPF, DMARC

Rules

- **DNS Verifications:** Ensure active DKIM, SPF, and DMARC settings match:

- DKIM tokens match Route53 configuration targets.
- SPF record: `"v=spf1 include:amazonses.com -all"`.
- DMARC record: `"v=DMARC1; p=none; rua=mailto:dmarc@takeseat.me; fo=1"`.

Edge Cases

- **Sandbox Sending Constraints:** In sandbox mode, emails can only be sent to verified addresses. Production migration is required to send emails to any recipient.
- **Access Denied Execution Error:** If backend Lambda executions fail to send messages, confirm that IAM execution policies grant `ses:SendEmail` and `ses:SendRawEmail` permissions.

Technical Notes

- View send statistics using:

```
aws cloudwatch get-metric-statistics --namespace AWS/SES --metric-name Send --dimensions Name=Environment,Value=prod --start-time 2024-01-01T00:00:00Z --end-time 2024-01-02T00:00:00Z --period 3600 --statistics Sum --region us-east-1
```

Related Documents

- [System Architecture Overview](#)
- [Security Overview](#)

 docs/troubleshooting/troubleshooting-guide.md

Troubleshooting Guide

Overview

This document provides debugging steps, logs verification procedures, and resolution actions for recurring infrastructure and application anomalies.

Responsibilities

- Diagnose failed messaging deliveries.
- Resolve database connection pool exhaustion.
- Debug web layouts and scroll behaviors.

Architecture / Flow

1. **Detect Issue:** Alert triggers or logs register error stack traces.
2. **Review Logs:** Query CloudWatch logs or local console terminal buffers.
3. **Verify Settings:** Check configuration parameters and database records.
4. **Apply Fix:** Execute runbook commands to recover service state.

Rules

- **Access Privilege Logs:** Never print raw user credentials, unmasked JWTs, or credit card numbers in diagnostic output files or application logs.

Edge Cases

- **Z-API Gateway Disconnected:** If guests are not receiving text notifications:

- Check the Z-API dashboard instance state.
- Scan the WhatsApp Web link QR code if the device has lost connection.
- Verify that the API token has not expired.
- **Database Connection Pool Exhaustion:** If API requests return HTTP 500 with `PrismaClientInitializationError` :
 - Verify that database connections are properly released.
 - Check if the Lambda function concurrency limit has exceeded the Aurora cluster's max connection capacity.
 - Increase the connection limit by appending `?` `connection_limit=15` to the `DATABASE_URL` parameter.

Technical Notes

- Tail Lambda execution logs using the AWS CLI command:

```
aws logs tail /aws/lambda/takeseat-api-prod --follow --  
region us-east-1
```

Related Documents

- [Email Service Runbook](#)
- [Local Setup Guide](#)

 docs/reference/api-reference.md

API Reference

Overview

This document catalogs the REST API endpoints exposed by the TakeSeat backend, specifying payload properties and HTTP response structures.

Responsibilities

- Document authentication route payloads.
- Detail operational waitlist control routes.
- Specify billing management callbacks.

Architecture / Flow

- All administrative requests must supply a valid authentication header:
`Authorization: Bearer <JWT_TOKEN>`
- All database-backed endpoints return JSON bodies.

Rules

Authentication

POST `/auth/login`

- **Description:** Authenticate operator credentials.
- **Request Body:**

```
{
  "email": "user@example.com",
  "password": "securepassword"
}
```

- **Response (200 OK):**

```
{
  "accessToken": "eyJhbG...",
  "refreshToken": "7f8...",
  "user": {
    "id": "usr_99",
    "email": "user@example.com",
    "role": "ADMIN"
  }
}
```

Waitlist Operations

POST `/queue`

- **Description:** Add a new guest party to the active waitlist.
- **Request Body:**

```
{
  "name": "John Doe",
  "phone": "+5511999999999",
  "partySize": 4,
  "whatsappOptIn": true,
  "isPriority": false,
  "notes": "Near window"
}
```

- **Response (201 Created):**

```
{
  "id": "entry_123",
  "name": "John Doe",
  "partySize": 4,
  "status": "WAITING",
  "publicToken": "token-xyz"
}
```

GET **/queue/:publicToken**

- **Description:** Fetch status metrics of a specific queue guest party. No Auth header required.
- **Response (200 OK):**

```
{
  "name": "John Doe",
  "status": "WAITING",
  "position": 3,
  "etaMinutes": 45
}
```

POST **/queue/:id/call**

- **Description:** Notify a guest party that their table is ready.
- **Response (200 OK):**

```
{
  "id": "entry_123",
  "status": "CALLED",
  "calledAt": "2026-05-20T00:00:00.000Z"
}
```

Billing

GET `/billing/status`

- **Description:** Check restaurant subscription plan status.
- **Response (200 OK):**

```
{
  "plan": "PRO",
  "subscriptionStatus": "TRIALING",
  "trialEndAt": "2026-05-27T00:00:00.000Z",
  "canAccessFeatures": true
}
```

Edge Cases

- **Expired Token Errors:** Expired JWT tokens will trigger an HTTP 401 Unauthorized response:

```
{ "error": "Token expired" }
```

Technical Notes

- REST API base path: `/api` .

Related Documents

- [Security Overview](#)
- [Authentication Flow](#)

 docs/infrastructure/infra-overview.md

Infrastructure Overview

Overview

TakeSeat infrastructure is designed as a cost-effective, high-availability Serverless architecture on Amazon Web Services (AWS), fully managed via Terraform Infrastructure as Code (IaC).

Responsibilities

- **Compute Hosting:** AWS Lambda runs backend API handlers and DB migration runners.
- **Relational Storage:** Amazon Aurora Serverless v2 MySQL manages data persistence.
- **Static Assets Delivery:** Amazon S3 hosts React frontend build files, fronted by Amazon CloudFront CDN.

Architecture / Flow

- **Request Routing Path:**
 1. Client operator access → queries CloudFront CDN → serves React files cached from S3 bucket.
 2. Client API request → routes to AWS API Gateway → triggers API Lambda execution.
 3. API Lambda → queries Aurora Serverless v2 DB cluster over MySQL protocol.
 4. Outbound API requests (e.g. Z-API WhatsApp notifications) → Lambda forwards directly to public internet routes.

Rules

- **Serverless Compute Mode:** All operations utilize lambda execution models. No persistent virtual servers (EC2 instances) are used.
- **Infrastructure as Code:** Any modifications to AWS resources (S3 buckets, security group settings, DNS routing) must be declared in Terraform configuration files located in `infra/terraform/`. Manual edits via AWS Web Console are prohibited.

Edge Cases

- **VPC Bypass for NAT Avoidance:** Backend Lambda functions run outside the private VPC network. This enables direct access to external APIs (Z-API WhatsApp gateway) and database clusters without incurring AWS NAT Gateway operational costs. Security is enforced via MySQL strong password encryption and strict database IP security groups.

Technical Notes

- Runtime platform: Node.js 20.x on Amazon Linux 2023.
- Database credentials: Sync is handled using AWS Secrets Manager.

Related Documents

- [Deployment Guide](#)
- [Backend Architecture](#)

 docs/infrastructure/deployment.md

Deployment Guide

Overview

TakeSeat deployments are fully automated using GitHub Actions workflows. Every commit pushed to the `main` branch triggers compilation, test, and release runs to production.

Responsibilities

- **Frontend Syncing:** Compile static assets and upload to S3, invalidating CloudFront caches.
- **Backend Packaging:** Transpile TypeScript files, compile architecture-specific dependencies, and zip assets.
- **Migration Orchestration:** Apply Prisma schema updates immediately after uploading function bundles.

Architecture / Flow

1. **GitHub Trigger:** Code is merged into `main` → triggers `.github/workflows/deploy-prod.yml`.
2. **Build Stage:**
 - Frontend runs `npm ci` and `vite build`. Static artifacts sync to the production S3 bucket. CloudFront cache is invalidated.
 - Backend runs `npm ci`. Decoupled binary dependencies like `bcrypt` are rebuilt for Linux architecture:

```
npm rebuild bcrypt --build-from-source
```

- TypeScript is compiled into Javascript CommonJS files under `/dist`.
3. **Lambda Deploy:** Zip bundles are uploaded to S3, updating the API Lambda code source.
 4. **Migration Execute:** The workflow calls the database migration Lambda (`takeseat-migrate-prod`), executing pending database migrations.

Rules

- **Environment Separation:** Settings, database passwords, and provider API tokens must never be committed to repository code files. These parameters must reside in GitHub Secrets and load as environment variables during build execution.
- **Schema Safety:** If a database schema change is non-backward compatible (destructive migrations), deployment must be scheduled during low-traffic windows.

Edge Cases

- **Bcrypt Compilation Failures:** If the runner fails to compile native C++ modules, pre-compiled binaries matching AWS Lambda CPU architectures must be loaded.

Technical Notes

- Production build packages are kept in AWS S3 buckets as deployment archives.

Related Documents

- [Infrastructure Overview](#)
- [Local Setup Guide](#)